

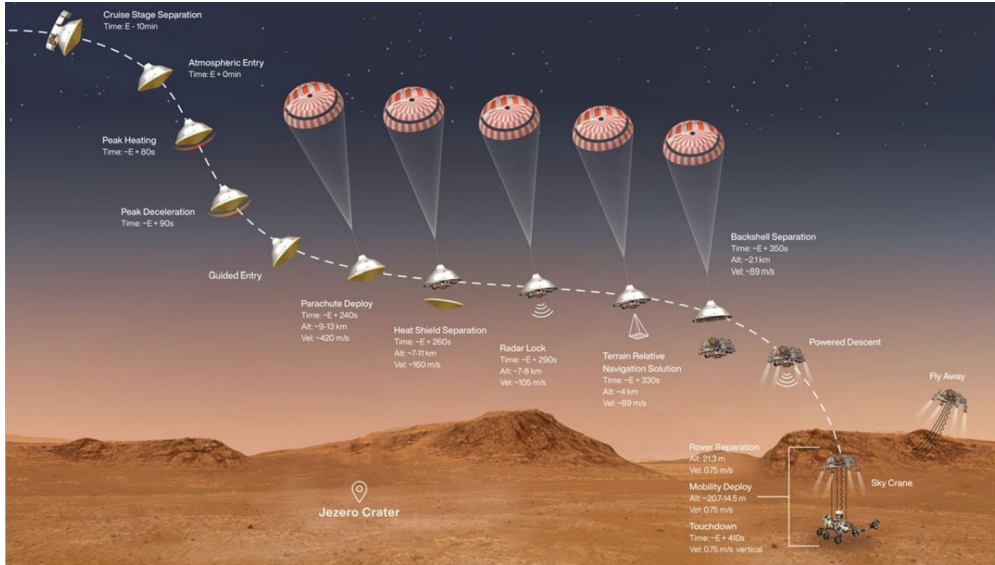


Figure 23.2 NASA JPL Mars lander. Image courtesy of NASA.

23.3. Landing concept of operations

In this chapter, we will focus on powered descent. This is the type of landing done on the Moon or other airless moons. The concept of operations (CONOPS) is shown in Fig. 23.3. The mission sequence has the following stages

1. Reorientation to the burn descent orientation;
2. Powered descent to the terminal point;
3. Hover at the terminal point to determine a safe landing spot;
4. Translation to a safe spot;
5. Vertical descent;
6. Touchdown and engine shutdown.

The landing sequence starts in a near-circular orbit near the surface. The parking orbit is assumed to be over the landing spot. Since the Moon rotates this is only true for a short period. The guidance algorithm needs to account for the surface movement relative to the inertial frame.

In this chapter, we will use a lunar lander as an example.

23.4. Selenographic coordinates

The selenographic frame is shown in Fig. 23.4. ϕ'_m is the Selenographic Latitude, which is the acute angle measured normal to the Moon's equator between the equator and

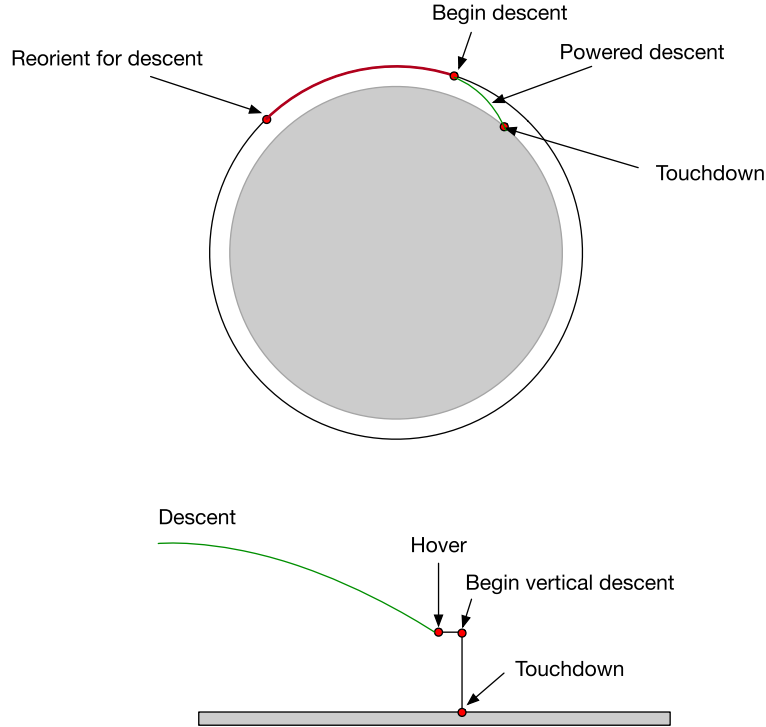


Figure 23.3 Lunar-landing concept of operations.

a line connecting the geometrical center of the coordinate system with a point on the surface of the Moon. The angle range is between -90 and $+90$ degrees. λ_m is the Selenographic Longitude, which is the angle measured towards the West in the Moon's equatorial plane, from the lunar prime meridian to the object's meridian. The angle range is between 0 and 360 degrees. North is the section above the lunar equator containing Mare Serenitatis. West is measured towards Mare Crisium.

The relationship to the ECI frame is shown in Fig. 23.5. i is the inclination between the Moon's equator and the ECI equator. Ω' is the longitude of the mean ascending node of the lunar orbit referenced to the mean equinox. ω is the angle between the ascending node and the lunar prime meridian.

The linear tangent guidance law is an analytical solution to the powered-descent problem assuming constant acceleration [1]. Since the mass of the lander decreases with time this assumes a variable thrust. The problem is posed as a two-dimensional flat planet problem with a uniform vertical gravity field. This is the same problem solved in [2].

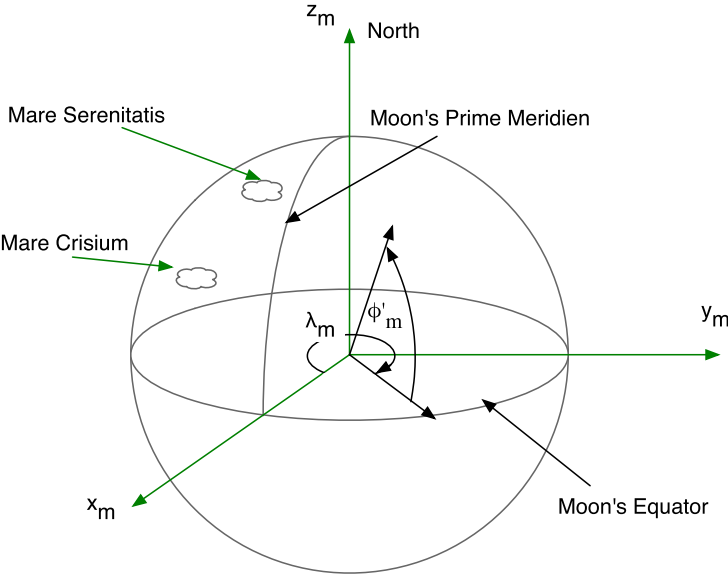


Figure 23.4 Selenographic frame.

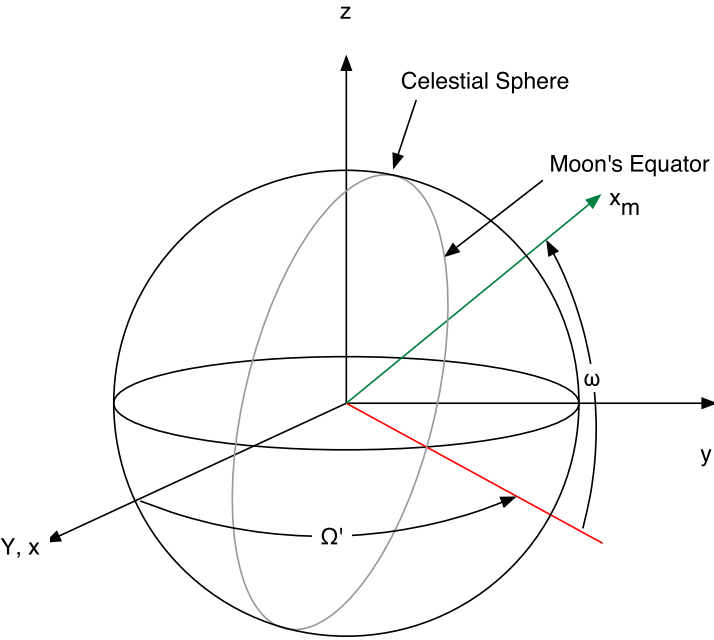


Figure 23.5 Selenographic to ECI frame.

23.5. Linear-tangent guidance law

The linear-tangent guidance law is the solution to the problem

$$\dot{s} = f(s, \beta) \quad (23.1)$$

where the state vector is

$$s = \begin{bmatrix} x \\ \gamma \\ u \\ v \end{bmatrix} \quad (23.2)$$

and the right-hand side is

$$f(s, \beta) = \begin{bmatrix} u \\ v \\ a \cos \beta \\ a \sin \beta - g \end{bmatrix} \quad (23.3)$$

where u is the horizontal velocity along x , v is the vertical velocity along γ , β is the thrust direction angle, a is the rocket acceleration, and g is the acceleration of gravity in the $-\gamma$ direction. a is assumed constant and the only control is β .

We want to solve the minimum-time descent problem. We do not care about the final value of x . Once we solve the problem, we make sure we start the final x distance from the landing point. The boundary conditions are

$$s(0) = \begin{bmatrix} 0 \\ h \\ U \\ 0 \end{bmatrix} \quad (23.4)$$

$$s(t_f) = \begin{bmatrix} \sim \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (23.5)$$

where $x(T)$ is not specified. We write the Hamiltonian as

$$H = L(s, \beta, t) + \lambda^T(t)f(s, \beta, t) \quad (23.6)$$

The control is found from

$$\frac{\partial H}{\partial \beta} = 0 \quad (23.7)$$

The cost is

$$J = \phi[s(t_f), t_f] + \int_0^{t_f} [H - \lambda^T f] dt \quad (23.8)$$

In our problem $L = 1$ and $\phi = 0$ so $J = t_f$. This makes the cost the time of flight.

The minimum-time problem is solving the two-point boundary value problem using the following state and costate equations

$$\dot{s} = f(s, \beta) \quad (23.9)$$

$$\dot{\lambda} = - \left(\frac{\partial f}{\partial s} \right)^T \lambda \quad (23.10)$$

$$0 = \left(\frac{\partial f}{\partial \beta} \right)^T \lambda \quad (23.11)$$

and the costate vector is

$$\lambda = \begin{bmatrix} \lambda_x \\ \lambda_y \\ \lambda_u \\ \lambda_v \end{bmatrix} \quad (23.12)$$

Three states are specified at the end. Since one is not specified, the corresponding costate must be zero when $t = t_f$. Thus the minimum time conditions are

$$\lambda_x(t_f) = 0 \quad (23.13)$$

$$(\lambda^T f)_{t_f} = -1 \quad (23.14)$$

The costate equations are

$$\dot{\lambda}_x = 0 \quad (23.15)$$

$$\dot{\lambda}_y = 0 \quad (23.16)$$

$$\dot{\lambda}_u = -\lambda_x \quad (23.17)$$

$$\dot{\lambda}_v = -\lambda_y \quad (23.18)$$

The control condition is

$$0 = \lambda_v \cos \beta - \lambda_u \sin \beta \quad (23.19)$$

Since λ_x is zero at $t = t_f$ it is always zero. λ_u is a constant that we can set to 1. Therefore the control is the linear tangent law

$$\tan \beta = \tan \beta_0 - ct \quad (23.20)$$

The problem is finding β_0 , c , and t_f . These are found by solving the differential equations for u , v , and γ . The differential equations are most easily solved by replacing t with β as the independent variable. Taking the derivative of the linear tangent law with respect to β gives

$$dt = -\frac{\sec^2 \beta}{c} d\beta \quad (23.21)$$

The equations for u , v , and γ are now

$$\frac{du}{d\beta} = -\frac{a}{c} \sin \beta \sec^2 \beta = -\frac{a}{c} \sec \beta \quad (23.22)$$

$$\frac{dv}{d\beta} = -\frac{a}{c} \sin \beta \sec^2 \beta = -\frac{a}{c} \sec \beta \tan \beta \quad (23.23)$$

$$\frac{d\gamma}{d\beta} = -\frac{v}{c} \sec^2 \beta \quad (23.24)$$

The effect of gravity is to add the particular solutions $-gt_f$ to the v solution and $-\frac{1}{2}gt_f^2$ to the γ solution. This gives us three equations in the three unknowns of β_0 , t_f , and c . The solution for v is

$$v = \frac{a}{c} (\sec \beta_0 - \sec \beta) \quad (23.25)$$

The third equation becomes

$$\frac{d\gamma}{d\beta} = -\frac{a}{c^2} \sec^3 \beta - \left(\frac{a}{c^2} \sec \beta_0 + gt_f \right) \sec^2 \beta \quad (23.26)$$

The three equations that need to be solved for the three constants are

$$\frac{Uc}{a} = \log |\sec \beta_f + \tan \beta_f| - \log |\sec \beta_0 + \tan \beta_0| \quad (23.27)$$

$$0 = \frac{a}{c} (\sec \beta_0 - \sec \beta_f) - gt_f \quad (23.28)$$

$$\begin{aligned} h = & \left(\frac{a}{c^2} \sec \beta_0 + gt_f \right) (\tan \beta_0 - \tan \beta_f) \\ & + \frac{a}{2c^2} (\sec \beta_0 \tan \beta_0 + \log |\sec \beta_0 + \tan \beta_0| - \sec \beta_f \tan \beta_f \\ & - \log |\sec \beta_f + \tan \beta_f|) - \frac{1}{2}gt_f^2 \end{aligned} \quad (23.29)$$

where

$$\tan \beta_f = \tan \beta_0 - ct_f \quad (23.30)$$

These cannot be solved analytically and must be solved using downhill simplex, `fminsearch`, in MATLAB®.

23.6. Lunar-lander model

A six-degrees-of-freedom model was built for training and testing terrain-relative navigation. It includes the rotational and translational degrees-of-freedom. The dynamical model is

$$T_t = I\dot{\omega} + \dot{I}\omega + \omega^\times I\omega \quad (23.31)$$

$$F_g + F_t = m\dot{v} \quad (23.32)$$

$$\dot{r} = v \quad (23.33)$$

$$\dot{m} = -\frac{F_t}{u_e} \quad (23.34)$$

$$\dot{I} = \dot{m}I_s \quad (23.35)$$

$$\dot{q} = g(q, \omega) \quad (23.36)$$

$$I_s = \frac{I(0)}{m(0)} \quad (23.37)$$

where q is the attitude quaternion, I is the inertia matrix, F_g is the gravitational force, F_t is the thruster force, m is the mass, u_e is the engine-exhaust velocity, T_t is the thruster torque, and ω is the angular rate. I_s is a constant scaling matrix that relates inertia to mass. Slosh is not modeled as the tanks are assumed to have baffles that damp fuel motion.

Fig. 23.6 gives images of landers. Many commercial companies and government organizations are also building and flying landers. The most recent, at the time of writing this book, is the Chinese Chang'E-5 probe that sent samples back to Earth.

The lunar-lander model is given in Table 23.1. We assume a bipropellant engine. The fuel load is computed from the rocket equation, modified to include tankage fraction.

Table 23.1 Lander parameters.

Parameter	Symbol	Value
Dry mass	$m_p + fm_f$	22 kg
Fuel mass	m_f	20.4 kg
Fuel-tankage fraction	f	0.1
Main engine-exhaust velocity	u_e	325 s
Descent acceleration multiple	n	3
Descent time	t_d	365.8 s
Hover time	t_h	120 s
Hover altitude	h_h	100 m

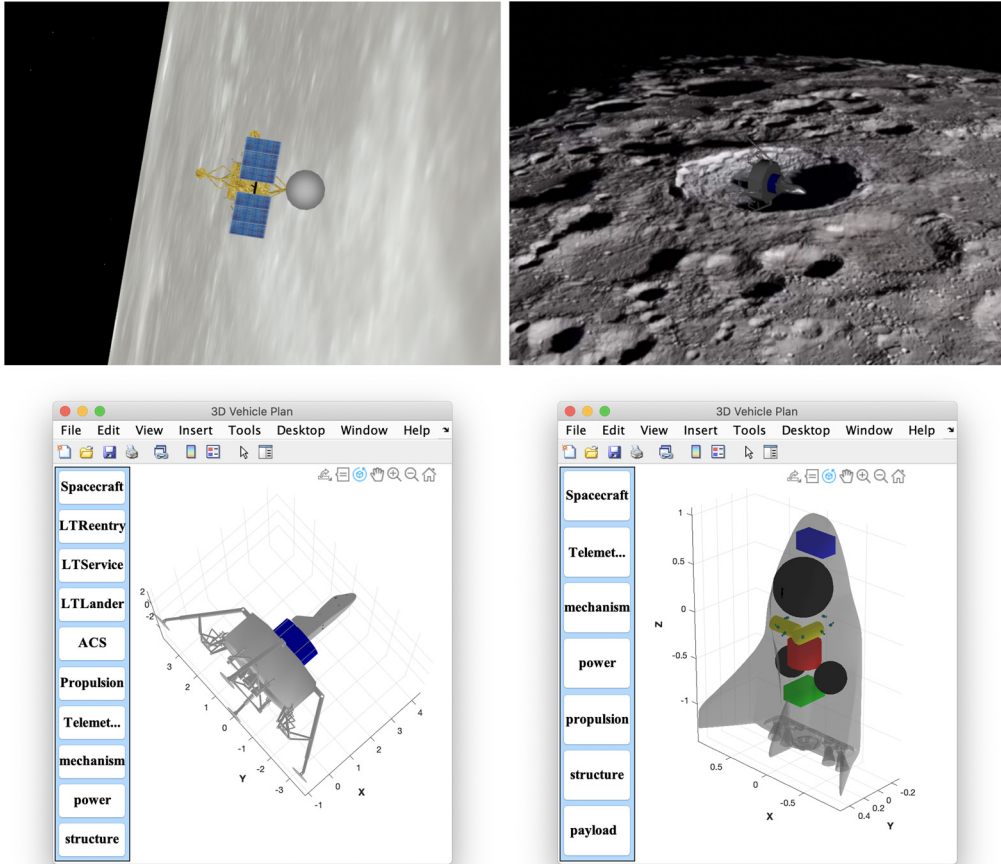


Figure 23.6 Proposed landers. The upper left is a lander with a high-gain antenna. The next three are a proposed commercial lunar helium-3 return vehicle. The reentry vehicle would be reusable. All of the avionics are in the reentry vehicle, include all RCS thrusters. Images courtesy of Princeton Satellite Systems.

$$\gamma = e^{\frac{\Delta u}{u_e}} \quad (23.38)$$

$$m_f = m_p \frac{\gamma - 1}{1 + (1 - \gamma)f}$$

where m_p is the mass of everything except the tanks and fuel, and f is the fuel-tankage fraction.

The total velocity change includes all the terms in Table 23.2.

This results in a dry mass of 22 kg and a fuel mass of 20.4 kg.

Table 23.2 Landing-velocity change contributions. g is the acceleration of lunar gravity, n is the descent control acceleration multiple, t_d is the descent time t_h is the over time, t_a is the bang-bang descent time, and t_s is the bang-band switch time.

Contribution	Equation	Value (m/s)
Linear tangent descent	ngt_d	1781
Hover	gt_h	194
Landing	ngt_a	16
Attitude control	5% of Δu fuel load	100
Total		2091

23.7. Optimal descent

In this chapter, we will use a single optimal solution for the powered descent. The descent will assume a constant deceleration at three times the acceleration of the lunar gravity. Example 23.1 shows the optimal trajectory. The first pair of plots shows the linear tangent law. The second pair shows the “optimal” trajectory using *fmincon*. The terminal altitude is 100 m where the hover will commence.

23.8. Descent control

Descent control requires that the thrust vector be aligned with the desired inertial vector. In addition, it is usually necessary that a landing sensor, such as radar, a navigation camera, or lidar, be pointed at the lunar surface. The required attitude quaternion is found in the two vectors. Assume that the sensor vector is the trajectory radius vector, n , and the thrust vector is u . Both vectors are defined in the ECI frame. Assume that the thrust vector is along x in the body frame. The quaternion must align x with u and bring the sensor vector, in the body frame, b , close to n . Fig. 23.7 shows the geometry.

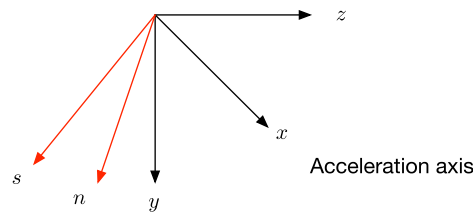
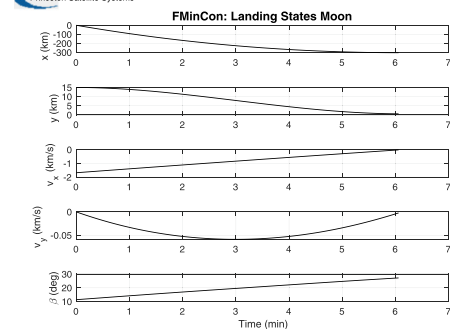
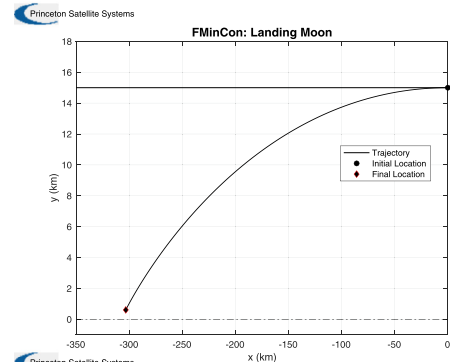
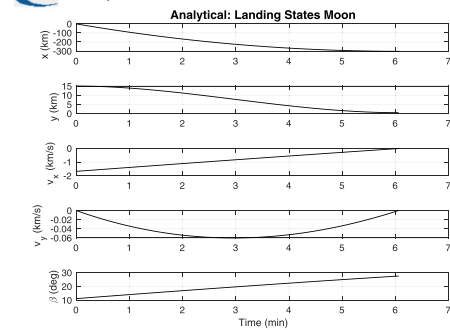
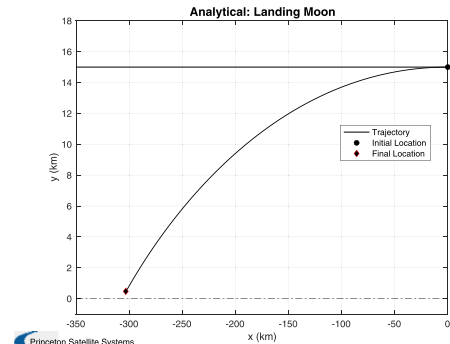


Figure 23.7 Aligning two vectors with ECI vectors. The acceleration vector is aligned exactly, the other as close as possible.

```

1
2
3 %% Parameters
4 muMoon = Constant('mu_moon');
5 rMoon = Constant('equatorial_radius_moon');
6 d = struct;
7 d.u = sqrt(muMoon/rMoon); % Orbit
8 d.g = muMoon/rMoon^2; % Gravity
9 d.h = 15; % Altitude (km)
10 d.n = 100; % Number of increments
11 d.a = 3e-3; % Engine acceleration
12 d.x = [0;d.h;-d.u;0]; % Initial state
13 algorithm = 'interior-point';
14
15 % Find the thrust direction angles
16 [beta, t, tMin] = BilinearTangentLaw(d.u, d.g,
    d.a, d.h, d.n);
17 fprintf(1, 'Analytical minimum time: %12.4s sec\n',
    tMin);
18
19 % We don't want the last beta since the vehicle
    is on the ground
20 beta = beta(1:end-1);
21 d.n = d.n-1;
22
23 % Do this to get a landing
24 beta = fliplr(beta);
25
26 % Simulate the landing
27 Simulate2DLanding(t, beta, d, 'Analytical');
28
29 %% Now repeat with fmincon
30
31 % fmincon options
32 opts = optimset('Display','iter-detailed', ...,
33                 'TolFun', 0.6, ...,
34                 'algorithm', 'algorithm', ...,
35                 'TolCon', 1e-5, ...,
36                 'MaxFunEvals', 100000);
37
38 d.hFinal = 0.6;
39
40 % The cost is time, which is a decision variable
41 % The cost is the time to reach the final state
    vector
42 costFun = @(x) LandingCost2D(x,d);
43
44 % The numerical integration of the state is in
    the constraint function
45 constFun = @(x) LandingConst2D(x,d);
46
47 % The final state vector is [x;0;0;0];
48 % We don't care what x is since we can always
    start the descent at the
49 % appropriate time.
50
51 % First guess for the time decision variable
52 dT = t(2:end) - t(1:end-1);
53
54 % Do this to get a reasonable first guess but not
    exact
55 beta = 1.2*beta;
56
57 % The decision variables are acceleration angle
    and time increment
58 x0 = [beta'; dT'];
59
60 % Lower and upper bounds
61 lB = zeros(length(x0),1);
62 uB = [(pi/2)*ones(length(dT),1); 100*ones(
    length(dT),1)];
63
64 % Find the optimal decision variables.
65 x = fmincon(costFun, x0, [], [], [], lB, uB,
    constFun, opts);
66
67 %% Set up the simulation
68 beta = x(1:d.n)';
69 dT = x(d.n+1:2*d.n);
70 t = zeros(1,length(dT));
71 for k = 2:d.n+1
72     t(k) = t(k-1) + dT(k-1);
73 end
74
75 % Simulate the landing
76 Simulate2DLanding(t, beta, d, 'FMinCon');
77
78 a = d.a;
79 h = d.h;
80 tDescend = t(end);
81 save('LandingTrajectory', 'beta', 't', 'h', 'a',
    'tDescend', 'tDescend');

```



Example 23.1: Lunar descent.

Let a be the acceleration unit vector.

$$z = a \times s \quad (23.39)$$

$$y = z \times a \quad (23.40)$$

$$m = \begin{bmatrix} x^T \\ y^T \\ z^T \end{bmatrix} \quad (23.41)$$

$$\theta = \tan^{-1} \frac{z^T n}{y^T n} \quad (23.42)$$

$$q = \begin{bmatrix} \cos \frac{\theta}{2} \\ a_x \sin \frac{\theta}{2} \\ a_y \sin \frac{\theta}{2} \\ a_z \sin \frac{\theta}{2} \end{bmatrix} \quad (23.43)$$

23.9. Terminal control

The terminal control uses a bang-bang translation controller for vertical control during terminal ascent. Horizontal control is via velocity damping. It is assumed that the lander is hovering above the landing spot.

Thrust-vector orientation is achieved by reorienting the vehicle or the engine nozzle. The reorientation controller needs to be fast enough so that the engine is pointing along the desired vector. It can set the throttle to zero until you reach the desired orientation.

The terminal descent control has four modes:

1. Mode 1 - Drive the vertical velocity to zero and hover;
2. Mode 2 - Drive all velocities to zero;
3. Mode 3 - Bang-bang vertical control to land;
4. Mode 4 - Engine shuts down.

Once it reaches Mode 4 the control system needs to set the throttle to zero. The first two are velocity-nulling controllers. The altitude-hold controller is used for both.

23.10. Altitude hold

The altitude-hold algorithm is a proportional-derivative (PD) controller in position and velocity. The control algorithms are

$$\epsilon = r - r_{set} \quad (23.44)$$

$$f = mK_h \begin{bmatrix} -\epsilon_x - \tau_h v_x \\ -\epsilon_y - \tau_h v_y \\ \epsilon_z + \tau_h v_z - g \end{bmatrix} \quad (23.45)$$

where m is the mass, g is the acceleration of gravity, aligned with z , K_h is the forward gain, τ_h is the damping time constant, and r_{set} is the position set point. This allows the vehicle to maneuver in three dimensions as needed. The thrust is

$$T = \frac{f}{|f|} \quad (23.46)$$

The thrust vector in the body frame is

$$T_V = \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} \quad (23.47)$$

If q is the quaternion that rotates T_V into f , then

$$u = q \otimes \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (23.48)$$

which is the vector that the attitude control system needs to track. Example 23.2 shows the altitude-hold control system.

23.11. Bang-bang landing algorithm

Given the gravitational acceleration, g , the position and velocity at time t_s are

$$z = h - \frac{1}{2}gt_s^2 \quad (23.49)$$

$$v = gt_s \quad (23.50)$$

where t_s is the switch time and h is the initial altitude. Given the rocket acceleration a , the time to reach zero velocity is

$$t_a = \frac{g}{a}t_s \quad (23.51)$$

The position must be zero at time t_a

$$h - \frac{1}{2}gt_s^2 - gt_s t_a + \frac{1}{2}at_a^2 = 0 \quad (23.52)$$

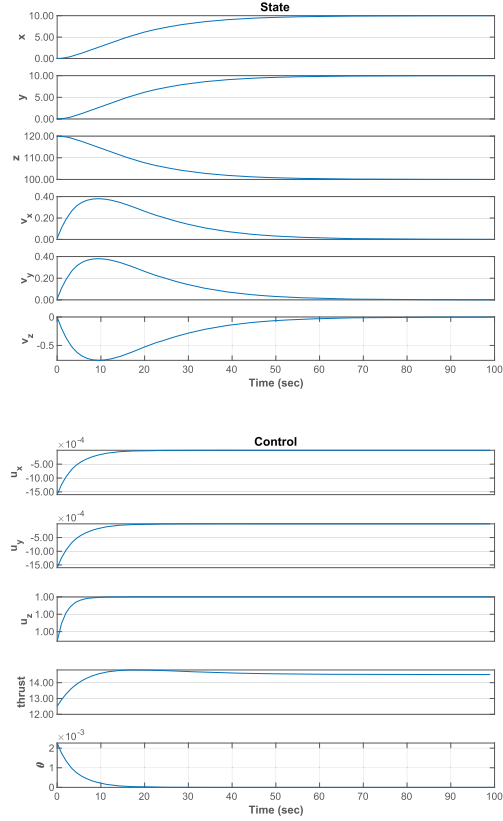
We can then solve for t_s

$$t_s = \sqrt{\frac{2h}{g(1 + \frac{g}{a})}} \quad (23.53)$$

```

1 mu = 4.9028e+03;
2 rM = 1738;
3 h = 100;
4 dT = 1;
5 n = 100;
6 g = 1000*mu/(rM+h)^2;
7 xP = zeros(11,n);
8 m = 10;
9 r = 1.2*h;
10 x = [0;0;r;0.01;0.01;0];
11 xSet = [10;10;h];
12 omegaN = 0.1;
13 tauH = 2/omegaN;
14 kH = omegaN^2;
15 thrust = 400;
16 for k = 1:n
17     [u,thrust,angle] = AltitudeHold(x,kH,tauH,g,m,
18                                     xSet);
19     xP(:,k) = [x;u;thrust;angle];
20     x = RK4(@RHS,x,dT,0,u*thrust/m,g);
21 end
22 t = (0:n-1)*dT;
23 yL = {'x' 'y' 'z' 'v_x' 'v_y' 'v_z'};
24 TimeHistory(t,xP(1:6,:),yL,'State')
25 TimeHistory(t,xP(7:11,:),{'u_x' 'u_y' 'u_z' '
    thrust' '\theta'},'Control')
26
27 %% Orbit equations
28 function xDot = RHS(x,~,accel,g)
29
30 r = x(1:3);
31 xDot = [x(4:6);accel-[0;0;g]];
32
33 end

```



Example 23.2: Attitude-hold controller performance. It moves the vehicle 10 meters in x and y while maintaining altitude, given a small initial altitude error.

This requires accurate knowledge of the thrust. This would be obtained from the accelerometers on the IMU. The velocity change is

$$\Delta u = at_a \quad (23.54)$$

23.12. Simulation results

Example 23.3 simulates the lander control system.

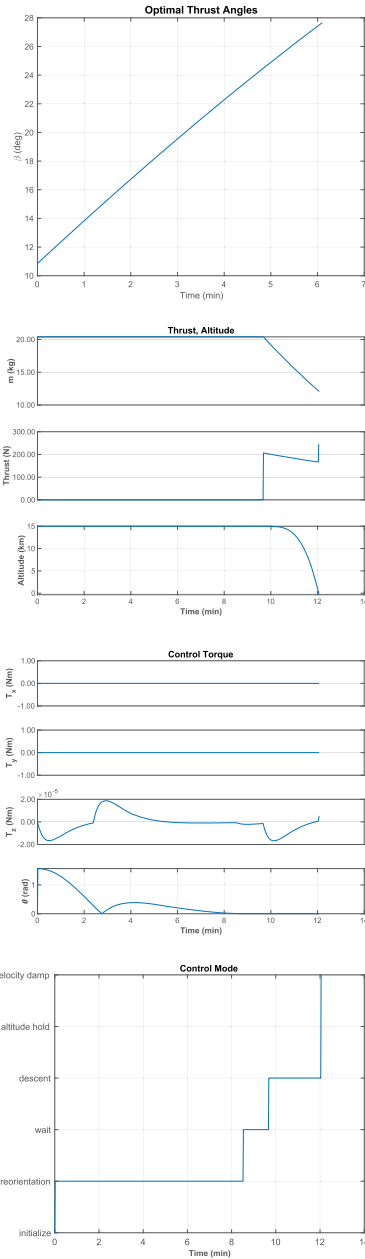
References

- [1] A.E. Bryson, Y.-C. Ho, Applied Optimal Control, Hemisphere, 1975.
- [2] B. Acikmese, S.R. Ploen, Convex programming approach to powered descent guidance for Mars landing, Journal of Guidance, Control, and Dynamics 30 (5) (2007) 1353–1366.

```

1 % Constants
2 secInDay      = 86400;
3 rMoon        = 1738;
4 mu           = 4.9028e+03;
5
6 % User inputs
7 dT           = 1; % sec
8 n            = 10000; % Number of steps
9 startDate    = [2023 5 1 0 0 0];
10 fuelMass    = 20.4;
11 dryMass     = 22;
12 uE          = 325*9.806;
13
14 % Set up the simulation
15 t           = (0:n-1)*dT;
16
17 % Set up the data structures
18 dRHS        = RHSLunarLander;
19 dRHS.uE     = uE;
20 dRHS.massDry = dryMass;
21 dLLC        = LunarLandingControl;
22 dLLC.traj    = load('LandingTrajectory.mat');
23
24 % Initial state and Julian date
25 dRHS.jD0    = Date2JD(startDate);
26 dLLC.jDTouchdown = dRHS.jD0...
27 + (dLLC.tReorientation+dLLC.traj.tDescend + 120)
28 /secInDay;
29 r = rMoon + dLLC.traj.h;
30 rV = [r;0;0;0;sqrt(mu/r);0];
31 x = [rV;dRHS.x0(7:end)];
32 x(14) = fuelMass;
33 jD = dRHS.jD0 + t/secInDay;
34
35 % Simulation loop
36 xP = zeros(length(x)+7,n);
37 for k = 1:n
38     [dLLC, dRHS] = LunarLandingControl( x, t(k),
39     dRHS, dLLC );
40     alt = Mag(x(1:3))-rMoon;
41     xP(:,k) = [x;dRHS.thrust;alt;dRHS.torque;...
42     dLLC.angError;dLLC.modeNumber];
43     x = RK4(@RHSLunarLander,x,dT,t(k),dRHS);
44     if( alt <= 0 )
45         break
46     end
47 end
48
49 % Plots
50 k = 1:3;
51 TimeHistory(t,xP(k,:),yL(k),'Position');
52 k = 4:6;
53 TimeHistory(t,xP(k,:),yL(k),'Velocity');
54 k = 7:10;
55 TimeHistory(t,xP(k,:),yL(k),'Quaternion');
56 k = 11:13;
57 TimeHistory(t,xP(k,:),yL(k),'Angular_Velocity');
58 k = [14 21 22];
59 TimeHistory(t,xP(k,:),yL(k),'Thrust_Altitude');
60 k = 15:20;
61 TimeHistory(t,xP(k,:),yL(k),'Inertia');
62 k = 23:26;
63 TimeHistory(t,xP(k,:),yL(k),'Control_Torque');
64 k = 27;
65 % Control mode display
66 TimeHistory(t,xP(k,:),yL(k),'Control_Mode');
67 s = {'initialize','reorientation','wait','descent',
68     'altitude_hold','velocity_damp','landing','shutdown'};
69 set(gca,'yticklabel',s,'ytick',1:6,'ylim',[1 6]);
70
71 Figui

```



Example 23.3: Lunar-descent control simulation.